ASSIGNMENT 2 – CO2

NAME:P.B.VAMSI KRISHNA
192525147

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 31/10/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/VAMSI00927/DSA0120--C-.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with

working code without changing the rest of the program, execute it and compare your output with the expected output.
Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Call by Value, by Reference and by Pointer 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to students' fees. A function written to apply the concession appears to work when its result is printed inside the function, but the fee stored for the student never changes.

Your Challenge

Write the concession function three ways – passing the fee by value, by reference and by pointer. Before running, predict the stored fee after each call, then verify.

Inquiry

Explain where each version stores its parameter, why only two of them change the caller's fee, and when a const reference parameter should be used instead.

Design Decision

Several concessions may apply to one student. Decide whether the function should change the fee in place or return the new fee, and defend your choice.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


void change(int a, int& b, int* c)
{
 a += 10;
```

```
 b += 10;
 *c += 10;

}


int& bigger(int& p, int& q) { return p > q ? p : q; }


void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }


int main()

{

 int a = 1, b = 2, c = 3;

 change(a, b, &c);

 std::cout << a << ' ' << b << ' ' << c << '\n';


 bigger(a, c) = 100;

 std::cout << a << ' ' << c << '\n';


 show(1);

 show(1, 2);

 show(1, 2, 3);

 std::cout << '\n';

}
```

My predicted output (write before running):
Explain which variables changed in each call and why, and which default arguments were used by each call to show().

Complete the Code

Complete TODO 1 below so that the caller's fee is changed, and execute the

program. #include <iostream>

```
void applyByValue(long long fee, int percent)
{
 fee -= fee * percent / 100; // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
 // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by
// reference)

}

int main()
{
 long long hostelFee = 45000;
 applyByValue(hostelFee, 10);
 std::cout << "after call by value : " << hostelFee << '\n';
 applyConcession(hostelFee, 10);
 std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:

```
after call by value : 45000
after call by reference: 40500
```

Explain why the call by value leaves the fee at 45000.

Alternative Concept and Complexity Analysis

If the concession function returned the new fee (long long applyConcession(long long fee, int percent)) instead of changing the fee through a reference, what would the solution be? Analyse the time (O(1) in all versions) and the space copied per call for a long long and for a

large Student object passed by value, by reference and by const reference.

Evidence Expected

Three implementations, predictions before execution, observed results and a defended interface design.

Q2 – Return by Reference and Its Dangers 4 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Examination Marks Register

The marks register keeps the marks of a class in an array. A function returns a reference to the highest mark so that a moderation of +2 can be applied directly to it. A second function returns a reference to a local variable.

Your Challenge

Implement both functions and show:

• How returning a reference lets the caller change the array element (and even use the call on the left of =)

• What the compiler warns about when a reference to a local variable is returned, and why using it is undefined behaviour

• The difference between returning a reference, returning a const reference and returning a copy

Changed Condition

The register is now stored in a std::vector that may grow. Explain why a reference obtained before the vector grows may become invalid.

Alternative Concept and Complexity Analysis

If the highest mark were returned as an index (int) or as a pointer (int*) instead of a reference (int&), what would the calling code look like? Analyse the time to find the highest mark (O(n) in all three) and the space (O(1)), and compare the safety of each when the array is later replaced by a growing vector.

Evidence Expected

Working functions, compiler-warning evidence, a correct explanation of dangling references and a justified response to the changed condition.

Q3 – Default Arguments and Function Overloading 6 Marks

Bloom’s Level: K5 – Evaluate

Problem Context – Courier Charge Calculator

A courier counter charges by weight; express delivery and insurance are optional. The developer wrote charge(double weight), charge(double weight, bool express) and charge(double weight, bool express = false, bool insured = false). The program no longer compiles.

Your Challenge

Explain the compiler error (ambiguous call). Then design the charge functions so

that: • Common calls are short (default arguments)

• Different kinds of input (weight in grams as int, weight in kg as double, a Parcel object) are handled by overloading

• No call is ambiguous

Before execution, predict which function each of six calls you design will select, including charge(2) and charge(2.0f).

Inquiry

Explain how the compiler chooses among overloaded functions (exact match, promotion, conversion), and why default arguments must be given from the right.

Alternative Concept and Complexity Analysis

If the courier charge took a single Parcel structure with optional fields (or a function template) instead of several overloaded functions with default arguments, what would the solution be? Analyse the time of overload resolution (at compile time, no run-time cost) and the space copied per call, and compare how each design handles a new option.

Evidence Expected

An explained ambiguity, a redesigned set of functions, predicted and observed selections, and a reasoned explanation of overload resolution.

Q4 – Specifying a Class with an Array Inside It 6 Marks

Bloom’s Level: K5 – Evaluate

Problem Context – Savings Account Mini-Statement

A bank wants each SavingsAccount object to remember its last 10 transactions for the mini statement printed at an ATM. Older transactions are discarded.

Your Challenge

Specify the class with private data (account number, balance, an array of 10 transactions and a count) and public member functions deposit(), withdraw() and miniStatement(). Define

some member functions inside the class and some outside using the scope resolution operator.

Inquiry

Explain why the array must be private, what happens when the 11th transaction arrives if you shift the array versus use a circular index, and the cost of each.

Design Decision

The bank now wants the last 50 transactions. Decide which design changes least and defend it.

Alternative Concept and Complexity Analysis

If the last 10 transactions were kept in a std::deque (push_back and pop_front), or in a circular buffer with a start index, instead of shifting a fixed array, what would the solution be? Analyse the time to add a transaction (O(k) for shifting k elements versus O(1)) and the space used in each design.

Evidence Expected

A complete class specification, member functions defined inside and outside the class, a justified handling of the 11th transaction and a defended design.

Q5 – Static Members and Arrays of Objects 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – College Bus Seat Tickets

The college bus has a fixed number of seats. Each BusTicket object must receive the next seat number, tickets beyond the capacity go to a waiting list, and the transport office must know how many seats have been issued without looking at any particular ticket.

Design Starting Point

You are given only the following principle:
Data that belongs to the whole class – not to any one object – should exist only once, however many objects are created.

Your Challenge

Convert this principle into a class design. Determine:

• Which members must be static, and where a static data member must be defined •

Why a static member function cannot use this or a non-static member • How an array

of BusTicket objects is created and initialised

• What happens to the count when tickets are created inside a loop or a function

Test Your Design

Create an array of tickets larger than the capacity and verify the seat numbers and the waiting list.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


class Token {
 static int next;
 int id;


public:
 Token() { id = ++next; }
 int get() const { return id; }
 static int issued() { return next - 100; }
};


int Token::next = 100;


int main()
{
 Token counter[3];
 Token extra;
 std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
<< ' ' << Token::issued() << '\n';
}
```

My predicted output (write before running):

Explain why the static counter starts at 100 and how many times the constructor ran. Complete the Code

Complete TODO 1 and TODO 2 below using the static members, and execute the program. #include <iostream>

#include <string>

```
class BusTicket {
 static int issued; // shared by all BusTicket objects
 static const int CAPACITY = 3;
 int seat;
 std::string name;

public:
 explicit BusTicket(const std::string& n) : seat(0), name(n)
 {
 // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
 // give this ticket the next seat number; otherwise leave seat as 0 (waiting  //
list)

 }

 static int count()
 {
 // TODO 2: Return the number of seats issued so far

 }

 void print() const
 {
```

```
 std::cout << name << ": "
 << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))  << '\n';
 }
};

int BusTicket::issued = 0; // definition of the static data member

int main()
{
 BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
BusTicket("Chitra"), BusTicket("Dev")};
 for (const BusTicket& t : trip) t.print();
 std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

Create the tickets inside a function that is called twice and explain how the count behaves. Alternative Concept and Complexity Analysis

If the seat counter were a global variable, or were kept in a separate TicketOffice object passed to every BusTicket, instead of a static data member, what would the solution be? Analyse the time (O(1) in all) and the space (one counter in all), and compare the encapsulation and the ability to run two buses at once.

Evidence Expected

A student-derived class design using static members, an array of objects, test evidence and a reasoned explanation of static members.

Q6 – Friend Functions and Inline Functions 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Train Journey Planner

A journey planner stores departure and arrival times in a Time class whose hours and minutes are private. It must compute the minutes between two times and decide which time is earlier, and these small functions are called millions of times while planning connections.

Your Challenge

Design the comparison. Determine:

• Why a friend function is needed when the function works on two objects equally

• How a friend function differs from a member function in how it is called and what it can access

• Whether the small functions should be inline, and what inline actually asks the compiler to do

• What would be lost if the private members were made public instead

Correctness Check

Test with times on either side of noon and the same time twice, and verify by

hand. Growth Investigation

Measure the time for 10,000,000 calls with and without inline (compile with -O0 and with - O2), and explain the result.

Deployment Decision

Decide whether the friend functions should stay or be replaced by member functions and getters, and defend your decision.

Complete the Code

Complete TODO 1 and TODO 2 below inside the friend functions, and execute the

program. #include <iostream>

```
class Time {
 int h, m;

public:
 Time(int hh, int mm) : h(hh), m(mm) {}
```

```
 friend int minutesBetween(const Time& a, const Time& b);
 friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
 // TODO 1: Using the private members h and m, return the absolute difference between
// the two times in minutes

}

bool earlier(const Time& a, const Time& b)
{
 // TODO 2: Return true if a is earlier in the day than b

}

int main()
{
 Time departure(9, 45), arrival(13, 10);
 std::cout << minutesBetween(departure, arrival) << ' '
 << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output:

205 false

Test the same time twice and a pair of times on either side of noon.

Alternative Concept and Complexity Analysis

If the comparison were written as member functions (a.minutesTo(b), a.isEarlier(b)) or with public getters instead of friend functions, what would the solution be? Analyse the time per call (the same after inlining, O(1)) and measure it at -O0 and -O2, and compare the encapsulation of the three designs.

Evidence Expected

A justified friend-function design, correctness tests, measured inline timings at two optimisation levels and an evidence-based decision.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13 Marks

Bloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select ONE program that shows a significant limitation as the data grows or the requirement changes.

You are not permitted simply to replace your code with a library facility and claim it is better. You must understand the problem first and then develop an appropriate redesign.
A. Diagnose the Limitation – 2 Marks

Using your own evidence, identify where the design fails or slows down and which characteristic of the original design causes it.

Evidence Expected

A precise diagnosis supported by evidence from your original

program. B. Define the Design Objective – 2 Marks

State a measurable objective for the redesign.

Evidence Expected

A clearly stated objective derived from the identified limitation.

C. Design the Revised Solution – 4 Marks

Explain what you have changed in the class or function design, which C++ feature you use, whether extra memory is needed, and any new trade-off. Provide a class diagram or pseudocode.

Evidence Expected

A coherent redesign directly addressing the diagnosed limitation.

D. Predict Before Implementing – 1 Mark

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate the Redesign – 2 Marks

Compare Original → Redesigned using the same tests and

measurements. Evidence Expected

Comparable evidence for both versions.

F. Defend the Final Design – 2 Marks

Evaluate whether the objective was achieved, new costs, and where the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly explain why.

Evidence Expected

A critical, evidence-based evaluation and a defensible conclusion.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

• Your assignment report containing the required evidence for Q1–Q7 •

C++ source code for all programs

• Predictions, outputs, timings and compiler messages used as evidence

Assessment Verification

Students may be asked to explain any part of their predictions, code, tests or results during a brief verification discussion/viva.

ANSWERS:

Q1 – CALL BY VALUE, BY REFERENCE AND BY POINTER (6 MARKS)

1.1 Concept and comparison

Call by value creates a separate parameter copy. Changes affect only that copy. Call by reference creates an alias for the caller's object, so assignment to the parameter changes the original. Call by pointer passes an address; dereferencing that address changes the pointed-to object. A pointer can also be null, so it must be checked before dereferencing.

Passing method	Parameter form	What the function receives	Changes caller?	Typical use
Value	long long fee	Independent copy of fee	No	Small values or intentional copy
Reference	long long& fee	Alias to caller's fee	Yes	Modify an existing object
Pointer	long long* fee	Address of caller's fee	Yes, after *fee	Optional object / explicit address semantics
Const reference	const Student& s	Read-only alias	No mutation through s	Large object read-only input

1.2 Three concession implementations

```
#include <iostream>
void byValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void byReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
bool byPointer(long long* fee, int percent) {
    if (fee == nullptr || percent < 0 || percent > 100) return false;
    *fee -= (*fee) * percent / 100;
    return true;
}
int main() {
    long long merit = 45000, sports = 45000, singleParent = 45000;
    byValue(merit, 10);
    byReference(sports, 15);
    byPointer(&singleParent, 25);
    std::cout << merit << '\\n' << sports << '\\n' << singleParent << '\\n';
}
```

Predicted output:

```
45000\n38250\n33750
```

Explanation: byValue changes only its local copy, leaving merit at 45000. byReference subtracts 15% of 45000 (=6750), leaving 38250. byPointer subtracts 25% (=11250), leaving 33750.

1.3 Predict the supplied program output

```
1 12 13
1 100
16 13 6
```

Detailed reasoning: Initially a=1, b=2, c=3. change(a,b,&c) receives a by value, b by reference and c by pointer. The local copy a becomes 11 but the caller's a remains 1; b becomes 12 and c becomes 13. The first output is therefore 1 12 13. Next bigger(a,c) returns a reference to the larger value. Since c=13 and a=1, the expression is an alias to c; assigning 100 changes c to 100. Thus the second output is 1 100. show(1) uses y=5,z=10 and prints 16; show(1,2) uses z=10 and prints 13; show(1,2,3) prints 6. Combined correct output:

```
1 12 13\n1 100\n16 13 6
```

1.4 Complete TODO 1

```
void applyConcession(long long& fee, int percent) {
    if (percent < 0 || percent > 100) return;
    fee -= fee * percent / 100;
}
```

Expected output:

```
after call by value : 45000\nafter call by reference: 40500
```

Call by value leaves the stored fee unchanged because the function receives a copy. Call by reference aliases hostelFee and updates the caller's object.

1.5 Return-value alternative and design decision

```
long long applyConcession(long long fee, int percent) {
    if (percent < 0 || percent > 100) return fee;
    return fee - fee * percent / 100;
}
// Use: hostelFee = applyConcession(hostelFee, 10);
```

For several concessions, returning a new fee makes the transformation explicit and easy to chain: fee=applyConcession(fee,10); fee=applyConcession(fee,15). In-place reference modification is useful when the function's purpose is specifically to update an existing record. Recommended design: return a new amount for a pure calculation, then store it deliberately; use reference when updating a Student record as one controlled operation.

Parameter design	Time	Extra parameter-copy space	Large Student object
By value	O(1) for fixed-size fee	O(1) for long long	Copies the complete object; can be expensive
By reference	O(1)	O(1) reference/alias	No Student copy
By const reference	O(1)	O(1) reference/alias	No copy; prevents modification through parameter

All three fee operations take O(1) time. Passing a large object by reference or const reference avoids copying its fields. A const reference is the best default for a large input that the function only needs to inspect.

Q2 – RETURN BY REFERENCE AND ITS DANGERS (4 MARKS)

2.1 Safe reference to an array element

```
#include <iostream>
```

```
int& highestMark(int marks[], int count) {
    int best = 0;
    for (int i = 1; i < count; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
int main() {
    int marks[] = {78, 91, 84, 88};
    highestMark(marks, 4) += 2;
    std::cout << highestMark(marks, 4) << '\\n';
    highestMark(marks, 4) = 100;
    std::cout << marks[1] << '\\n';
}
```

Output:

```
93\n100
```

The function returns an alias to the actual array element. Therefore +=2 and assignment on the left-hand side update the stored array element.

2.2 Dangling reference – unsafe example

```
int& unsafeValue() {
    int local = 50;
    return local; // ERROR: reference outlives local object
}
```

The local variable is destroyed when the function returns. The returned reference then refers to an object whose lifetime has ended. Accessing it is undefined behaviour; output cannot be predicted. Compilers commonly issue a warning such as 'reference to local variable returned' or 'returning reference to temporary', depending on compiler and exact code. Never use the dangling reference.

2.3 Reference, const reference and copy

Return form	Caller can modify result?	Lifetime / safety
int&	Yes	Safe only while the referenced object remains alive and at a stable address.
const int&	No through this reference	Same lifetime requirement; const does not extend lifetime of a returned local.
int	Modifies only the returned copy	Safe value independent of source lifetime.

2.4 Vector growth and alternative index/pointer

A std::vector may reallocate its storage when capacity is exceeded. Reallocation moves elements to a new memory block and invalidates references, pointers and iterators to old elements. A returned reference must not be retained across an operation that may reallocate the vector.

```
int highestIndex(const std::vector<int>& marks) {
    int best = 0;
    for (int i = 1; i < static_cast<int>(marks.size()); ++i)
        if (marks[i] > marks[best]) best = i;
    return best;
}
// Usage:
int index = highestIndex(marks);
marks[index] += 2;   // index remains a position after vector growth,
// but confirm index is still within the current size.
```

An index is safer across reallocation because it identifies a position rather than a memory address. It can still become logically wrong if elements are inserted, erased or reordered before that position. A pointer has the same invalidation risk as a reference.

Return choice	Find maximum time	Extra space	Vector reallocation safety
Reference	O(n)	O(1)	Invalidated by reallocation
Index	O(n)	O(1)	Survives reallocation; position may change logically
Pointer	O(n)	O(1)	Invalidated by reallocation
Copy	O(n)	O(1) for int	Independent value; cannot update original directly

Q3 – DEFAULT ARGUMENTS AND FUNCTION OVERLOADING (6 MARKS)

3.1 Why the original functions are ambiguous

The declarations charge(double), charge(double,bool) and charge(double,bool=false,bool=false) overlap. A call charge(2.0) can match all three because the second and third overloads can omit their defaulted arguments. The compiler sees multiple viable candidates and cannot choose a unique best function. Default arguments do not create a separate overload; they allow a call to omit trailing parameters.

3.2 Unambiguous design

```
#include <iostream>
struct Parcel {
    double weightKg;
    bool express = false;
    bool insured = false;
};
double charge(int grams, bool express = false, bool insured = false) {
    double kg = grams / 1000.0;
    return kg * 50.0 + (express ? 40.0 : 0.0) + (insured ? 20.0 : 0.0);
}
double charge(double kg, bool express = false, bool insured = false) {
    return kg * 50.0 + (express ? 40.0 : 0.0) + (insured ? 20.0 : 0.0);
}
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}
int main() {
    std::cout << charge(2) << '\\n';
    std::cout << charge(2.0) << '\\n';
    std::cout << charge(2.0f) << '\\n';
    std::cout << charge(2000, true) << '\\n';
    std::cout << charge(2.0, false, true) << '\\n';
    Parcel p{3.0, true, true};
    std::cout << charge(p) << '\\n';
}
```

Predicted overload selection and output:

Call	Selected overload	Calculation / result
charge(2)	charge(int, bool, bool)	2 grams = 0.002 kg; Rs 0.10
charge(2.0)	charge(double, bool, bool)	2 kg; Rs 100.00

charge(2.0f)	charge(double, bool, bool)	float promotes to double; Rs 100.00
charge(2000,true)	charge(int, bool, bool)	Rs 100 + Rs 40 = Rs 140.00
charge(2.0,false,true)	charge(double, bool, bool)	Rs 100 + Rs 20 = Rs 120.00
charge(p)	charge(const Parcel&)	3 kg + express + insurance = Rs 210.00

The six outputs are 0.1, 100, 100, 140, 120 and 210 respectively. The overload resolution for charge(2.0f) selects the double overload because float-to-double is a floating-point promotion, which is preferred to a conversion to int.

3.3 Overload resolution rules

- Exact match is preferred over promotion and conversion.
- A promotion (for example float to double, or char to int) is generally preferred over a standard conversion.
- If two candidates are equally good, the call is ambiguous and compilation fails.
- Default arguments are supplied only after a candidate function has been selected; they do not make one overload inherently better.
- Default arguments must be specified from right to left because callers can omit only trailing arguments.

3.4 Alternative Parcel-only design and complexity

```
struct ParcelRequest {
    double weightKg{};
    bool express{};
    bool insured{};
};
double calculate(const ParcelRequest& p) {
    return p.weightKg * 50.0 + (p.express ? 40.0 : 0.0)
         + (p.insured ? 20.0 : 0.0);
}
// Usage: calculate({2.0, true, false});
```

A Parcel structure centralises options and scales well when new options such as fragile handling or delivery confirmation are added. Overload resolution happens at compile time and adds no runtime cost. Passing Parcel by const reference avoids copying the structure. A by-value Parcel would copy its fields, which is still O(1) for this small structure. The overload design gives concise calls for different input types; the Parcel design gives one explicit request object and avoids a growing list of parameters.

Q4 – SPECIFYING A CLASS WITH AN ARRAY INSIDE IT (6 MARKS)

4.1 SavingsAccount class design

```
#include <iostream>
#include <string>
class SavingsAccount {
private:
    std::string accountNumber;
```

```
    long long balancePaise;
    long long transactions[10];
    int count;
    int nextIndex; // next position to overwrite when full
public:
    SavingsAccount(const std::string& number, long long opening)
        : accountNumber(number), balancePaise(opening),
          transactions{}, count(0), nextIndex(0) {}
    bool deposit(long long amountPaise) {
        if (amountPaise <= 0) return false;
        balancePaise += amountPaise;
        record(amountPaise);
        return true;
    }
    bool withdraw(long long amountPaise);
    void miniStatement() const;
private:
    void record(long long amountPaise) {
        transactions[nextIndex] = amountPaise;
        nextIndex = (nextIndex + 1) % 10;
        if (count < 10) ++count;
    }
};
bool SavingsAccount::withdraw(long long amountPaise) {
    if (amountPaise <= 0 || amountPaise > balancePaise) return false;
    balancePaise -= amountPaise;
    record(-amountPaise);
    return true;
}
void SavingsAccount::miniStatement() const {
    std::cout << "Account: " << accountNumber
              << " Balance (paise): " << balancePaise << '\\n';
    int start = (count == 10 ? nextIndex : 0);
    for (int i = 0; i < count; ++i)
        std::cout << transactions[(start + i) % 10] << '\\n';
}
```

Some functions (constructor, deposit and record) are defined inside the class; withdraw() and miniStatement() are defined outside using SavingsAccount:: scope resolution. Transactions are stored as signed paise: positive for deposit and negative for withdrawal.

4.2 Why private data and how the 11th transaction is handled

The transaction array must be private so outside code cannot change the balance history, corrupt the count or write beyond the array boundary. Public methods validate deposits and withdrawals and update balance and history together.

Method	When 11th transaction arrives	Time to insert	Space
Shift array	Move entries 1–9 to positions 0–8, then store new item at 9; oldest is discarded.	O(k), here O(10)	O(10)
Circular index	Overwrite oldest at nextIndex and advance modulo 10.	O(1)	O(10)
std::deque	push_back(new); if size>10 pop_front().	O(1) amortized operations at ends	O(10) retained items

4.3 Extension to 50 transactions

For a fixed requirement of 50, change the array size and modulo constant from 10 to 50. A named constexpr capacity avoids repeating the number. A circular buffer changes least because the same algorithm works with a different capacity. A deque is preferable if capacity changes dynamically, but the assignment asks for an internal array and a circular index demonstrates the underlying data structure clearly.

```
static constexpr int HISTORY_CAPACITY = 50;
long long transactions[HISTORY_CAPACITY];
// nextIndex = (nextIndex + 1) % HISTORY_CAPACITY;
```

For capacity k, shifting costs O(k) per new transaction; circular indexing costs O(1) per insertion. Both store O(k) transaction values. For k=10 or 50 the absolute shift cost is small, but circular indexing scales better as k grows.

Q5 – STATIC MEMBERS AND ARRAYS OF OBJECTS (7 MARKS)

5.1 Static member design

The issued-ticket count and seat capacity belong to the bus-ticket class as a whole, not to an individual ticket. Therefore issued and CAPACITY are static. The seat number and passenger name belong to each individual BusTicket object and remain non-static.

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }
    static int count() { return issued; }
    void print() const {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting")) << '\\n';
    }
};
int BusTicket::issued = 0;
int main() {
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\\n';
}
```

Expected output:

```
Asha: Seat 1\nBala: Seat 2\nChitra: Seat 3\nDev: Waiting\nIssued: 3
```

5.2 Predict the supplied Token program

```
101 103 104 4
```

The static counter starts at 100 because Token::next is initialised once outside the class. The three-element array constructs three Token objects, assigning IDs 101, 102 and 103. The extra object is the fourth constructor call and receives 104. issued() returns next−100 = 4.

5.3 Complete TODO 1 and TODO 2

```
BusTicket::BusTicket(const std::string& n) : seat(0), name(n) {
    if (issued < CAPACITY) {
        ++issued;
        seat = issued;
    }
}
static int count() {
    return issued;
}
```

In the supplied class, the constructor body and count() body above are inserted in their corresponding locations. The expected output is the five-line result shown in Section 5.1.

5.4 Static function, arrays and lifetime

A static member function has no this pointer because it is not called for one particular object. It can directly access static members, but cannot directly access a non-static member such as seat or name because no particular ticket object has been identified. It can access non-static data only through an explicitly supplied object.

An array is created with BusTicket trip[4] or initialised with four constructor expressions as shown. Each array element is a separate object and its constructor runs once. A static counter is shared across all objects and is not reset when a local array or function returns.

5.5 Function called twice and alternative designs

```
void issueBatch(const std::string names[], int n) {
    for (int i = 0; i < n; ++i) {
        BusTicket t(names[i]);
        t.print();
    }
}
// Calling issueBatch twice does not reset the static issued count.
```

If the first call issues two seats and the second call creates two more tickets, only the remaining capacity is available; the counter persists for the lifetime of the program. A reset should be an explicit, authorised operation rather than an accidental consequence of function return.

Counter design	Time	Storage	Encapsulation / multiple buses
Static member	O(1)	One counter per class	Good for one shared bus; awkward for two independent buses
Global variable	O(1)	One global counter	Weak encapsulation; any code can modify it
TicketOffice object	O(1)	One counter per office/bus	Best for multiple buses; each office owns its own capacity and count

For one bus, static members are simple. If two buses operate independently, a TicketOffice object passed to each ticket is safer because each office has its own issued count and capacity. The class can then represent multiple buses without sharing one global static counter.

Q6 – FRIEND FUNCTIONS AND INLINE FUNCTIONS (8 MARKS)

6.1 Time class and friend functions

```
#include <iostream>
#include <cstdlib>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
inline int minutesBetween(const Time& a, const Time& b) {
    int x = a.h * 60 + a.m;
    int y = b.h * 60 + b.m;
    return std::abs(x - y);
}
inline bool earlier(const Time& a, const Time& b) {
    return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main() {
    Time departure(9,45), arrival(13,10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\\n';
}
```

Expected output: 205 false. 09:45 is 585 minutes after midnight; 13:10 is 790 minutes after midnight; the difference is 205. arrival is not earlier than departure, so false.

6.2 Why friend and how it differs from member

A friend function is not a member of the class, but the class grants it access to private and protected members. It is useful when an operation treats two objects symmetrically, such as comparing a and b. It is called as minutesBetween(a,b), not a.minutesBetween(b). A member function is called on one object and receives an implicit this pointer. A friend has no this pointer and must receive the objects explicitly.

Design	Call syntax	Private access	Encapsulation
Friend function	earlier(a,b)	Direct access because class grants friendship	Private data remains private to other code
Member function	a.isEarlier(b)	Direct access as a member	Strong object-centred interface
Public getters	a.hour(), a.minute()	No direct private access; uses public interface	Private fields retained; getters expose selected data

6.3 Inline meaning and deployment decision

The inline keyword permits a function definition in multiple translation units under the One Definition Rule and suggests that the compiler may substitute the function body at the call site. It does not force the compiler to inline. Modern optimising compilers make this decision based on optimisation level, function size and call context. A small function may be inlined even without the keyword.

Keep the friend functions if comparison is naturally symmetric and direct private access makes the interface clear. Prefer member functions if the operation is conceptually owned by one Time object. Prefer getters when many external algorithms need read-only time values. Do not make h and m public merely to avoid writing friend functions; doing so lets unrelated code create invalid times such as 27:90.

6.4 Additional correctness tests

Test	Inputs	Expected minutesBetween	Expected earlier(a,b)
Across noon	11:55, 12:10	15	true
Same time	14:20, 14:20	0	false
Reverse order	18:00, 07:30	630	false
Morning to afternoon	09:45, 13:10	205	true

The given implementation assumes both times are valid 24-hour clock values. A production constructor should reject hour outside 0–23 and minute outside 0–59.

6.5 Inline performance experiment

```
#include <chrono>
#include <iostream>
volatile long long sink = 0;
inline int fastAdd(int a, int b) { return a + b; }
int normalAdd(int a, int b) { return a + b; }
int main() {
    constexpr int N = 10000000;
    auto start = std::chrono::steady_clock::now();
    long long total = 0;
    for (int i=0; i<N; ++i) total += fastAdd(i, 1);
    auto mid = std::chrono::steady_clock::now();
    sink = total;
    for (int i=0; i<N; ++i) total += normalAdd(i, 1);
    auto end = std::chrono::steady_clock::now();
    std::cout << "inline ms: "
      << std::chrono::duration<double,std::milli>(mid-start).count() << '\\n';
    std::cout << "normal ms: "
      << std::chrono::duration<double,std::milli>(end-mid).count() << '\\n';
}
```

Compile and compare using g++ -std=c++17 -O0 -Wall -Wextra timing.cpp -o timing_O0 and g++ -std=c++17 -O2 -Wall -Wextra timing.cpp -o timing_O2. Run both executables and record the observed milliseconds. Do not invent results: timings depend on processor, compiler, optimisation, thermal state and background tasks. At -O2, the compiler may inline both functions, so the difference can be negligible. At -O0, a call overhead may be more visible, but exact results must be measured.
Both versions perform O(1) work per call and O(N) total work for ten million calls. The function call does not change asymptotic complexity. Inline substitution can reduce call overhead but may increase code size if used excessively.

Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

Selected program: Q4 SavingsAccount transaction history

This program is selected because a shifting fixed array performs O(k) data movement whenever a new transaction arrives after the history is full. As the required history grows from 10 to 50 or more entries, the number of copied elements per insertion increases. The program remains correct, but repeated shifting is unnecessary.

A. Diagnose the limitation (2 marks)

Original feature	Limitation	Evidence / reasoning
Fixed array of 10	Capacity is hard-coded	Changing history size requires editing array declaration and modulo/loop limits.
Shift-on-full approach	Moves up to k−1 values for every new transaction	For 10 entries, up to 9 moves; for 50, up to 49 moves per insertion.
Coupled count/index logic	Easy to make off-by-one errors	The oldest/newest ordering must be handled consistently.

B. Define measurable objective (2 marks)

Objective: retain the latest k=50 transactions, discard only the oldest when full, preserve chronological mini-statement output, and reduce the insertion operation from O(k) element movement to O(1) index updates. Correctness must be verified for fewer than 50 transactions, exactly 50, and at least 51 transactions.

C. Design the revised solution (4 marks)

Use a fixed-capacity circular buffer. Store a fixed array of 50 transactions, the number currently stored, and the index where the next transaction will be written. When full, overwrite the oldest entry. The array uses O(k) memory; insertion uses O(1) time and does not allocate memory.

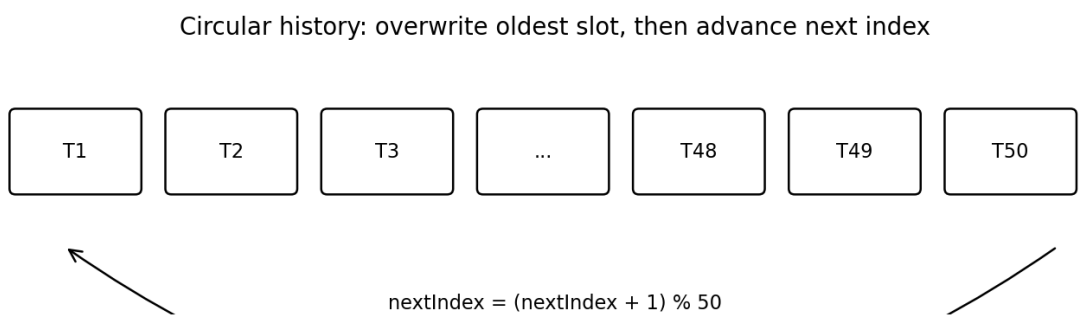


Figure: Circular buffer layout. After the buffer is full, the next insertion replaces the oldest item and the logical print order begins at nextIndex.

```
#include <array>
#include <iostream>
#include <string>
class TransactionHistory {
    static constexpr std::size_t CAPACITY = 50;
    std::array<long long, CAPACITY> values{};
    std::size_t count = 0;
    std::size_t nextIndex = 0;
public:
    void add(long long amountPaise) {
        values[nextIndex] = amountPaise;
        nextIndex = (nextIndex + 1) % CAPACITY;
        if (count < CAPACITY) ++count;
    }
    void print() const {
        std::size_t start = (count == CAPACITY) ? nextIndex : 0;
        for (std::size_t i = 0; i < count; ++i)
            std::cout << values[(start + i) % CAPACITY] << '\\n';
    }
    std::size_t size() const { return count; }
};
```

D. Predict before implementing (1 mark)

Input transactions	Expected retained count	Expected history
1 to 5	5	1,2,3,4,5
1 to 50	50	1 through 50
1 to 51	50	2 through 51 (transaction 1 discarded)
1 to 55	50	6 through 55 (transactions 1–5 discarded)

Prediction: insertion remains O(1) regardless of whether the history contains 5 or 50 transactions. The print operation remains O(k) because it must display each retained transaction.

E. Implement and validate (2 marks)

Use identical transaction sequences for the original shifting version and the circular version. Record insertion duration for a large number of insertions (for example, 1,000,000), and verify the retained history exactly matches the expected newest 50 values. Record actual measurements from your computer.

Test	Expected circular result	Pass condition
5 inserts	5 values in order	All 5 retained in chronological order
50 inserts	50 values in order	Buffer full, no loss
51 inserts	2–51	Oldest one removed
55 inserts	6–55	Oldest five removed
Repeated wrap-around	Last 50 inserted values	Correct order after multiple wraps

F. Defend final design (2 marks)

Measure	Shift array	Circular buffer
Insertion time	O(k) when full	O(1)
Memory	O(k)	O(k)
Chronological display	Simple after shift	Requires start-index calculation
Implementation	Straightforward but repeated copying	Slightly more index logic

Best use	Very small k and rare updates	Frequent updates and larger retained history

Conclusion: the circular buffer achieves the objective by eliminating shifting and keeping constant-time insertion while retaining exactly the most recent 50 transactions. The original shifting array may still be preferable for a very small history when implementation simplicity matters more than insertion cost. The redesigned solution does not reduce memory asymptotically; it improves the insertion operation and keeps predictable fixed storage.

APPENDIX – EXECUTION AND GITHUB SUBMISSION CHECKLIST

- Create repository DSA01-<RegisterNumber>-Assignment2.
- Create folders Q1, Q2, Q3, Q4, Q5, Q6 and Q7.
- Save each complete C++ program in its corresponding folder.
- Compile using g++ -std=c++17 -Wall -Wextra -pedantic filename.cpp -o program.
- Run every program and save genuine terminal screenshots showing the commands and output.
- Write predicted output before executing the program, as required by the assignment.
- Capture the compiler warning for the intentionally unsafe dangling-reference example; do not execute or dereference the returned dangling reference.
- Run the inline timing test at both -O0 and -O2 and record actual measurements with system/compiler details.
- Run the Q7 shifting and circular-buffer versions using the same data and record measured insertion times.
- Add README.md with student name, register number, compiler version, compile/run instructions and short result for each question.
- Commit progressively, at least one meaningful commit per question.
- Share the repository with faculty or set its visibility as instructed; paste the working URL on the cover page.